

Embedded Systems Architectures - Laboratory Guide 1

ESP32, ESP-IDF Toolchain

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Familiarization with a ESP32 board (ESP32-C6-DevKitC-1)
- Installation and familiarization with the ESP-IDF framework and support tools (command line and the VS Code extension)
- Creation, compilation and execution of simple sample projects

2. ESP32 Development Kit and Set of Components used in the Classes

Laboratory classes rely on the ESP32-C6-DevKitC-1 module from Espressif Systems to develop embedded systems and IoT solutions. This module has advanced connectivity features (Wi-Fi, Bluetooth LE, Zigbee and Thread), providing a platform to create a wide range of applications. In the context of this course, the ESP32-C6-DevKitC-1 module is supplied with a basic kit consisting of the following set of components:

- ESP32-C6-DevKitC-1 development board
- TC74 temperature sensor
- DHT20 temperature and humidity sensor
- USB-A to USB-C cable
- 0.96" RGB TFT display with SD card reader
- 16 GB SD card

Additionally, it is assumed that Students already own the following accessories:

- Prototyping board (breadboard)

- Cutting pliers
- Stripping pliers or tweezers
- Two oscilloscope probes

2.1 ESP32-C6-DevKitC-1 Board

The ESP32-C6-DevKitC-1 is based on the ESP32-C6-WROOM-1(U) module, a general-purpose module with 8 MB of SPI flash. Most of the I/O pins are available on the board's bottom headers, facilitating peripheral connections. Development with this board can be done by mounting the ESP32-C6-DevKitC-1 on a breadboard and connecting peripherals to it. A view of the board is shown in Figure 1.



Figure 1 - ESP32-C6-DevKitC-1 view.

Figure 2 depicts the structure of the ESP32-C6-DevKitC-1. It integrates Wi-Fi 6 functionality in the 2.4 GHz band, Bluetooth 5, and IEEE 802.15.4 (Zigbee 3.0 and Thread 1.3). All available General-Purpose Input/Output (GPIO) pins (except the SPI bus for the flash) are provided on the board headers. The board also features a 5V to 3.3V low-dropout (LDO) voltage regulator, a power-on LED, and a USB<->UART converter that supports transfer rates up to 3 Mbps.

The ESP32-C6-DevKitC-1 has a USB Type-C port compatible with USB 2.0, capable of transferring up to 12 Mbps. This port powers the board, writes firmware, communicates with the module using USB protocols, and supports Joint Test Action Group (JTAG) debugging. The board also includes a download (Boot) button, a reset button, and an addressable RGB LED driven by GPIO8. There is even a J5 connector

used for current measurement. Figure 3 identifies these elements on the board. The ESP32-C6-DevKitC-1 headers allow access to digital signals, serial communication interfaces such as UART, I2C and SPI, as well as analog pins for reading sensors. There are also dedicated pins for special functions, such as Pulse Width Modulation (PWM) for motor and servo control, as well as general-purpose inputs and outputs (GPIO) that can be programmed as needed. All pin functions are shown in Figures 4 and 5. More information about the board is available at: <https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32c6/esp32-c6-devkitc-1/index.html>.

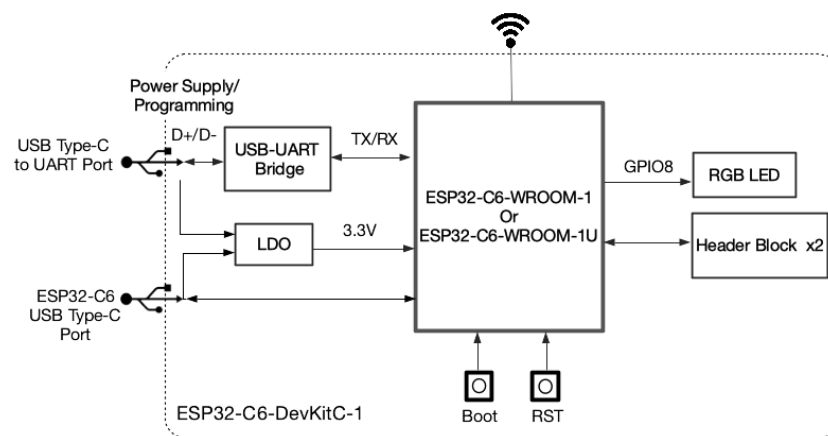


Figure 2 – Structure of the ESP32-C6-DevKitC-1.

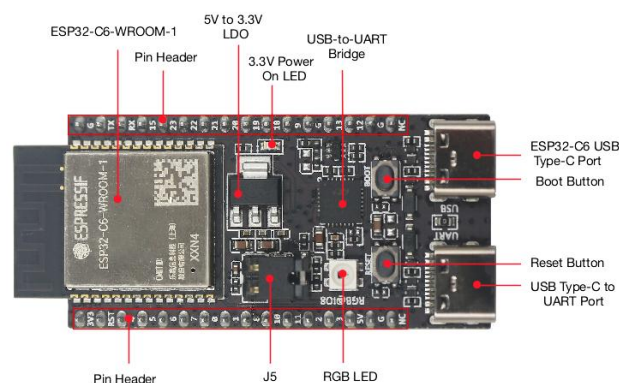


Figure 3 – Fundamental elements of the ESP32-C6-DevKitC-1.

ESP32-C6-DevKitC-1

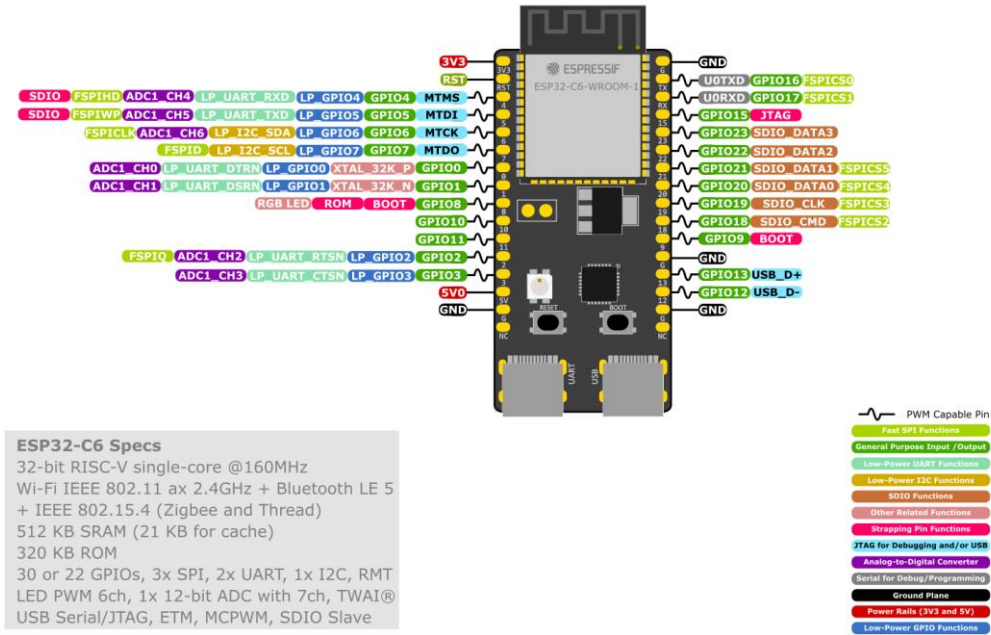


Figure 4 - Pin specifications for the ESP32-C6-DevKitC-1 (graphical view).

J1				J3			
No.	Name	Type ¹	Function	No.	Name	Type	Function
1	3V3	P	3.3 V power supply	1	G	G	Ground
2	RST	I	High: enables the chip; Low: disables the chip.	2	TX	I/O/T	U0TXD, GPIO16, FSPIC50
3	4	I/O/T	MTMS ³ , GPIO4, LP_GPIO4, LP_UART_RXD, ADC1_CH4, FSPIC51	3	RX	I/O/T	U0RXD, GPIO17, FSPIC51
4	5	I/O/T	MTDI ³ , GPIO5, LP_GPIO5, LP_UART_TXD, ADC1_CH5, FSPIC52	4	15	I/O/T	GPIO15 ³
5	6	I/O/T	MTCK, GPIO6, LP_GPIO6, LP_I2C_SDA, ADC1_CH6, FSPIC53	5	23	I/O/T	GPIO23, SDIO_DATA3
6	7	I/O/T	MTDO, GPIO7, LP_GPIO7, LP_I2C_SCL, FSPIC54	6	22	I/O/T	GPIO22, SDIO_DATA2
7	0	I/O/T	GPIO0, XTAL_32K_P, LP_GPIO0, LP_UART_DTRN, ADC1_CH0	7	21	I/O/T	GPIO21, SDIO_DATA1, FSPIC55
8	1	I/O/T	GPIO1, XTAL_32K_N, LP_GPIO1, LP_UART_DSRN, ADC1_CH1	8	20	I/O/T	GPIO20, SDIO_DATA0, FSPIC56
9	8	I/O/T	GPIO8 ^{2 3}	9	19	I/O/T	GPIO19, SDIO_CLK, FSPIC57
10	10	I/O/T	GPIO10	10	18	I/O/T	GPIO18, SDIO_CMD, FSPIC58
11	11	I/O/T	GPIO11	11	9	I/O/T	GPIO9 ³
12	2	I/O/T	GPIO2, LP_GPIO2, LP_UART_RTSN, ADC1_CH2, FSPIC59	12	G	G	Ground
13	3	I/O/T	GPIO3, LP_GPIO3, LP_UART_CTSN, ADC1_CH3	13	13	I/O/T	GPIO13, USB_D+
14	5V	P	5 V power supply	14	12	I/O/T	GPIO12, USB_D-
15	G	G	Ground	15	G	G	Ground
16	NC	-	No connection	16	NC	-	No connection

Figure 5 - Pin specifications for the ESP32-C6-DevKitC-1 (table view).

3. ESP-IDF Platform (Compilation Toolchain)

The ESP-IDF (Espressif IoT Development Framework) is a popular platform for developing IoT solutions on the ESP32-C6-DevKitC-1 module, providing a comprehensive development environment aligned with the module's capabilities. ESP-IDF is designed to leverage the ESP32-C6's features, offering a complete set of drivers, libraries, and examples. Its integration allows development to focus on the core of embedded and IoT applications, avoiding the low-level complexities of configuring and controlling peripherals. Additionally, this platform is open-source and reasonably well-documented, with an active community of users and developers that facilitates the discovery of solutions, troubleshooting, and the adoption of proven practices throughout the development cycle. Another significant advantage of ESP-IDF is its native support for advanced connectivity features, including Wi-Fi 6, Bluetooth 5, and low-power network protocols such as Zigbee and Thread. This integrated approach greatly simplifies deploying IoT applications that require access to multiple communication technologies.

3.1 Installation and Configuration

The first step in your journey with the ESP32-C6-DevKitC-1 module is to install ESP-IDF, which will allow you to edit the IoT application firmware, compile the source code, program the module, and perform debugging and testing. To do so, you must strictly follow the instructions provided on the Get Started page for the ESP32-C6 module from Espressif, where you will find a step-by-step guide on how to install the platform: <https://docs.espressif.com/projects/esp-idf/en/v5.5.2/esp32c6/get-started/index.html>. Independently, if you are installing on MS Windows, Linux, or macOS, you must install it for command-line (terminal) use as well as for use with the VS Code Extension (after installing the base ESP-IDF, also install the VS Code extension with the same name).

The following options must be selected in the installation process:

- ESP32-C6 device support
- Packages for building ESP-IDF documentation

- Packages for IDE support in ESP-IDF
- CMake build system
- Ninja build system

All prerequisites and device drivers must also be installed.

The following options do not need to be selected during the installation process:

- Packages for supporting debugging from a web browser
- Packages for CI with pytest
- Packages for specific test scripts
- Packages for ESP-IDF CI scripts

4. Testing of ESP-IDF Get Started Projects

The following subsections describe the major steps for compiling and running two example projects provided with ESP-IDF to test two board functionalities: terminal output and digital outputs. Both must be compiled and tested using both the command line and the VS Code Integrated Development Environment with the ESP-IDF extension installed.

4.1 Hello World Application

After installing the development environment, the next step is to create a basic application to test the entire ecosystem.

Create a copy in your home folder of the “Hello World” example located in:

“C:\Espressif\frameworks\esp-idf-v5.5.2\examples\get-started\hello_world”

(MS Windows path)

Then compile, flash and execute the example in the ESP32-C6 board. Finally, carefully analyse the source code against the application's output.

The following commands must be executed in the terminal, in the project root folder (after copying the example):

idf.py set-target esp32c6 (to select a particular device family)

idf.py build (to compile and link the application)

idf.py -p <SERIAL PORT> flash (to write the application image into the kit FLASH memory)

idf.py -p <SERIAL PORT> monitor (to open the terminal to visualize the application output)

The following command can also be used to configure the project settings:

idf.py menuconfig (if the project settings are changed, a complete build must be performed)

Repeat the procedure of “Hello World” project creation, compilation, and test inside the VS Code using the ESP-IDF extension (which must be installed prior to project creation).

4.2 Blink Application

Repeat the steps of the previous section for the “Blink” example to test the basic kit's I/O functionality (both in the command line and in VS Code).

Create a copy in your home folder of the “Blink” example

“C:\Espressif\frameworks\esp-idf-v5.5.2\examples\get-started\blink”

(MS Windows path)

Then compile, flash and execute the example in the kit. Finally, carefully analyse the source code against the application's output.